

Building a React Component to Consume a REST API

Aim

The aim of this exercise is to build a small React component that consumes data from a REST API using an AI coding assistant. The component should handle the main application states, such as loading the data, displaying the fetched data, and showing an error message if the API request fails.

For this implementation, I used **React**, **JavaScript**, **Vite**, **Axios**, **useState**, **useEffect**, and the **Fake Store REST API**. The API provides sample product data, so it was suitable for testing the component without creating a separate backend.

Tools and Technologies Used

- React
- JavaScript
- Vite
- Axios
- REST API
- `useState`
- `useEffect`
- AI coding assistant
- Fake Store API

REST Endpoint Used

The REST endpoint used in this exercise was:

`https://fakestoreapi.com/products`

This endpoint returns product information in JSON format. The response contains fields such as product ID, title, price, category, description, and image.

AI Coding Assistant Prompt

I used the following prompt with the AI coding assistant:

"Create a small React component in src/App.jsx that consumes the REST API <https://fakestoreapi.com/products> using axios and useEffect. Store the returned products in useState. Include a loading state that displays 'Loading products...' while the request is in progress and an error state that displays a suitable error message if the API request fails. After a successful request, display the product title and price in a simple list. Keep the component simple and do not add unnecessary dependencies, styling, or extra components."

After the AI generated the code, I reviewed the output instead of directly accepting it. I checked whether the API URL was correct, whether `useEffect` was used properly, and whether both loading and error states were handled.

React Component



```
import { useEffect, useState } from "react";
import axios from "axios";

function App() {
  const [products, setProducts] = useState([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setIsLoading(true);
    setError(null);

    axios
      .get("https://fakestoreapi.com/products")
      .then((response) => {
        setProducts(response.data);
      })
      .catch((err) => {
        setError(err);
      })
      .finally(() => {
        setIsLoading(false);
      });
  }, []);

  if (isLoading) {
    return <p>Loading products...</p>;
  }

  if (error) {
    return <p>Something went wrong while loading the products.</p>;
  }

  return (
    <div>
      <h1>Product List</h1>

      {products.map((product) => (
        <div key={product.id}>
          <h3>{product.title}</h3>
          <p>Price: ${product.price}</p>
        </div>
      ))}
    </div>
  );
}

export default App;
```

How the Component Works

The component starts with an empty product array using `useState`. I also created two additional state values, `isLoading` and `error`, to keep track of the current API request status.

The `useEffect` hook is used to make the REST API request when the component loads. Since the dependency array is empty, the request runs when the component is initially rendered.

I used `axios.get()` to send a GET request to the Fake Store API. When the request is successful, the response data is stored in the `products` state using `setProducts()`.

While the request is running, `isLoading` remains true and the application displays:

"Loading products..."

If the API request fails, the error is stored in the `error` state and the component displays an error message instead of leaving the page blank.

Finally, the `finally()` block changes `isLoading` to false after the request finishes. This means the application can move from the loading state to either the successful data state or the error state.

Testing the Loading State

To test the loading state, I refreshed the application and checked the page while the API request was being processed.

The application displayed:

Loading products...

After the API response was received, the loading message disappeared and the product information was displayed.

If the request is very fast, the loading message may only appear for a short time. Network throttling can be used in the browser's developer tools to make the loading state easier to observe.

Testing the Error State

I also tested the error handling by temporarily changing the API URL to an invalid endpoint.

For example:

```
https://fakestoreapi.com/does-not-exist
```

After refreshing the application, the request failed and the component displayed:

Something went wrong while loading the products.

I then changed the URL back to the correct products endpoint:

```
https://fakestoreapi.com/products
```

The application successfully loaded the product list again.

AI-Generated Code Review

I did not treat the AI coding assistant's output as final without checking it. I reviewed the generated component and verified the important parts manually.

The main points I checked were:

1. The REST API URL was correct.
2. `axios.get()` was used for the GET request.
3. `useEffect()` was used for the API call.
4. `useState()` was used to store the products.
5. A loading state was included.
6. An error state was included.
7. `setIsLoading(false)` was called after the request completed.
8. The returned product data was displayed using `.map()`.
9. Each product had a unique React `key`.
10. No unnecessary dependencies or extra functionality were added.

One important thing I noticed while reviewing the code was that Axios already provides the parsed response body through `response.data`. Therefore, there is no need to use `response.json()` as would normally be done with the browser's `fetch()` API.

Result

The React component was successfully created using an AI coding assistant. It consumes a REST API using Axios and displays the returned product information.

The application correctly handles the three important states:

Loading → API request in progress

Success → Product data displayed

Error → Error message displayed

This exercise helped me understand how React components can interact with REST APIs and how `useState` and `useEffect` can be combined with Axios to manage API data. It also gave me practical experience in reviewing AI-generated code and checking whether the generated implementation actually satisfies the requirements.

Conclusion

Through this exercise, I learned how to build a simple React application that communicates with a REST endpoint. Using an AI coding assistant made the initial implementation faster, but reviewing the generated code was still important to make sure the API call, state management, loading state, and error handling were implemented correctly.

The exercise also showed me that AI-generated code should be treated as assistance rather than something that can be accepted without verification.

AI Coding Assistant Prompt Transcript

Prompt 1 — Initial Component

My prompt:

"Create a small React component in `src/App.jsx` that consumes the REST API <https://fakestoreapi.com/products> using `axios` and `useEffect`. Store the returned products in `useState`. Include a loading state that displays 'Loading products...' while the request is in progress and an error state that displays a suitable error message if the API request fails. After a successful request, display the product title and price in a simple list. Keep the component simple and do not add unnecessary dependencies, styling, or extra components."

AI assistant response:

The AI assistant generated a React component using `useState`, `useEffect`, and `Axios`. It made a GET request to the Fake Store API, stored the response in state, and added loading and error handling.

Prompt 2 — Review and Correction

My prompt:

"Review the generated React component. Make sure the loading state is displayed while the API request is running, the error state is displayed when the request fails, and the product list is displayed only after a successful request. Do not add unnecessary features or dependencies."

My review:

I checked the generated code manually and verified the API URL, Axios request, state variables, `useEffect`, loading state, error state, and product rendering. I also checked that Axios uses `response.data` instead of `response.json()`.

Final Observation

The AI coding assistant helped generate the basic structure of the component, but I reviewed the code and tested the different states before considering the implementation complete.